

[Menu](#)

Mapping System

▼ Relevant source files

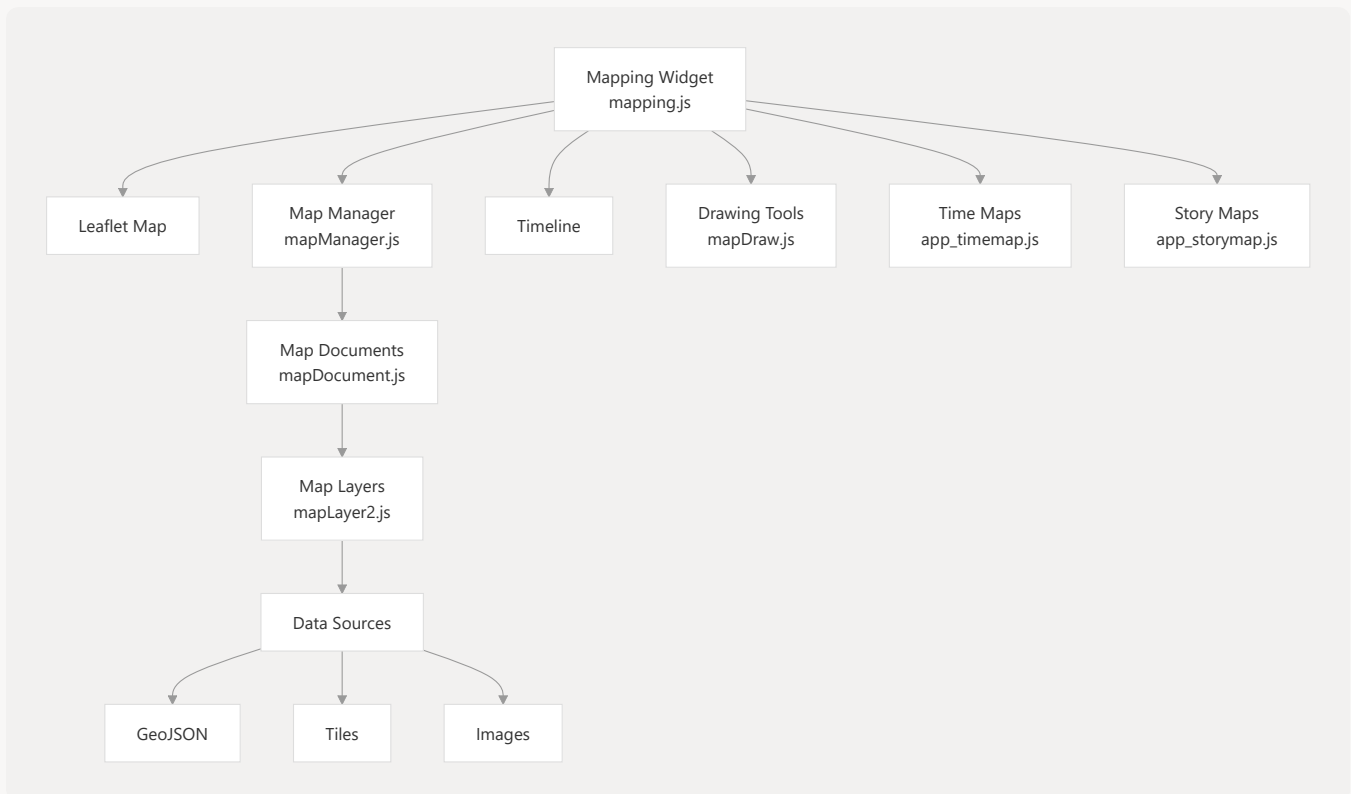
`hclient/framecontent/infoPage.php``hclient/framecontent/initPageLogin.php``hclient/widgets/cms/editCMS_WidgetCfg.html``hclient/widgets/cms/editCMS_WidgetCfg.js``hclient/widgets/search/searchInput.js``hclient/widgets/viewers/app_storymap.js``hclient/widgets/viewers/app_timemap.js``hclient/widgets/viewers/recordListExt.js``viewers/map/mapDocument.js``viewers/map/mapDraw.php``viewers/map/mapLayer2.js``viewers/map/mapManager.js``viewers/map/mapPreview.php``viewers/map/mapping.js`

The Mapping System is a core feature of the Heurist platform that provides geographical visualization capabilities. It allows users to display, interact with, and analyze spatial data through an intuitive map interface. This system is built on the Leaflet mapping library and includes components for managing map documents, layers, base maps, and interactive drawing tools.

For information about the related CMS Website Generator that can incorporate maps into websites, see [CMS Website Generator](#).

Architecture Overview

The Mapping System consists of several key components that work together to provide a complete mapping solution:



Sources: `viewers/map/mapping.js` 1-1016 `viewers/map/mapManager.js` 1-107
`viewers/map/mapDocument.js` 1-156 `viewers/map/mapLayer2.js` 1-196

Core Components

Mapping Widget

The Mapping Widget (`mapping.js`) is the primary interface for the mapping system. It wraps the Leaflet library and manages all map interactions. This widget:

- Creates and manages the map view
- Handles events like clicks, zooms, and pans
- Provides methods for adding and manipulating layers
- Integrates with other components like the timeline and drawing tools

The widget is initialized with various options that control its behavior:

```

mapping = $('#map_container').mapping({
  element_map: '#map',
  layout_params: {
    notimeline: false,
    nocluster: false,
    controls: 'legend,bookmark,geocoder',
  }
});
  
```

```
    basemap: 'OpenStreetMap'  
  },  
  oninit: onMapInit,  
  ondrawend: onMapDraw  
});
```

Sources: `viewers/map/mapping.js` 115-194 `viewers/map/mapping.js` 297-357

Map Manager

The Map Manager (`mapManager.js`) provides a user interface for managing map documents and layers. It creates a control panel that includes:

- Result Sets: Current search results
- Map Documents: Available map documents
- Base Maps: Available base maps (OpenStreetMap, Esri, etc.)

The Map Manager enables users to toggle layer visibility, change the base map, and create new map documents.

Sources: `viewers/map/mapManager.js` 118-267 `viewers/map/mapManager.js` 394-411

Map Documents

Map Documents (`mapDocument.js`) are containers for map layers. Each map document includes:

- Metadata (title, owner, etc.)
- Geographical extent (the area covered by the document)
- A collection of layers
- Style information

Map documents can be loaded from the database or created dynamically from search results.

Sources: `viewers/map/mapDocument.js` 44-97 `viewers/map/mapDocument.js` 154-189

Map Layers

Map Layers (`mapLayer2.js`) represent individual data layers within a map document. They handle:

- Loading data from various sources
- Applying styles and symbology
- Controlling visibility based on zoom levels
- Responding to user interactions

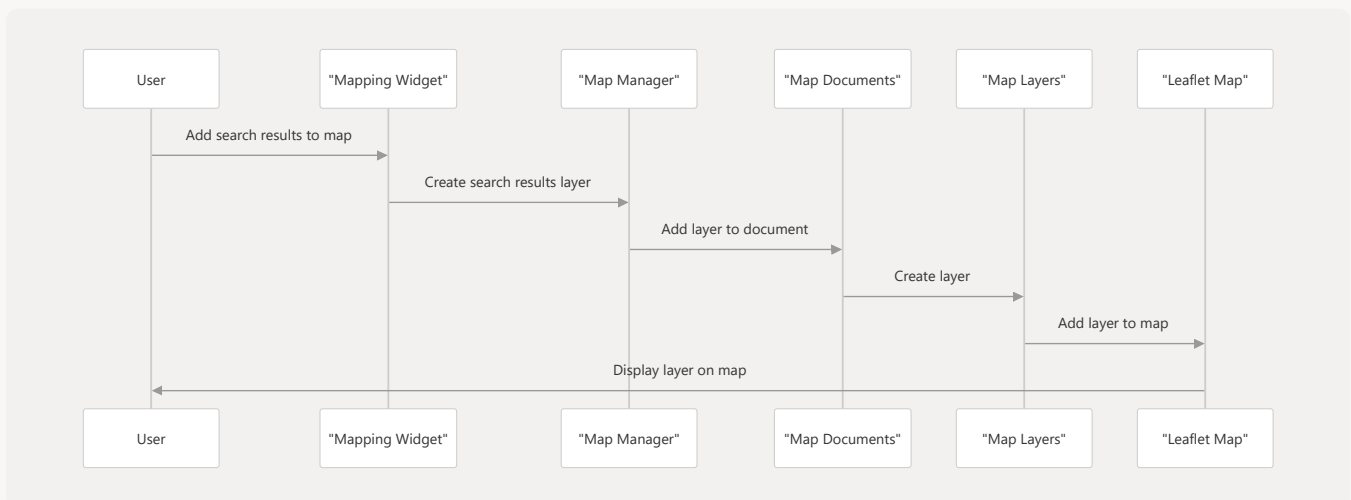
The system supports several types of layers:

- GeoJSON layers (points, lines, polygons from Heurist records)
- Tile layers (from services like OpenStreetMap)
- Image layers (georeferenced images)
- GeoTIFF layers
- KML, SHP, and other formats

Sources: `viewers/map/mapLayer2.js` 25-197 `viewers/map/mapLayer2.js` 447-507

Data Flow

The following diagram shows how data flows through the Mapping System when adding search results to a map:



Sources: `viewers/map/mapping.js` 845-848 `viewers/map/mapManager.js` 456-463
`viewers/map/mapDocument.js` 161-193

Key Features

Base Maps

The system supports a wide variety of base maps:

- OpenStreetMap
- OpenTopoMap
- Esri World maps (Street, Topo, Imagery, Relief)
- Stamen maps (Toner, Terrain, Watercolor)
- DARE Roman Empire historical map

- And many others

Users can select different base maps from the Map Manager control panel. The system also supports custom base maps and map filtering.

Sources: `viewers/map/mapping.js` 254-291 `viewers/map/mapping.js` 732-837

Layer Management

The system provides robust layer management capabilities:

- Add/remove layers
- Toggle layer visibility
- Adjust layer opacity
- Apply filters to layers
- Set layer order
- Control layer visibility by zoom level

Layers are organized in the Map Manager control panel, where users can interact with them.

Sources: `viewers/map/mapping.js` 957-996 `viewers/map/mapManager.js` 535-678

Drawing Tools

The system includes tools for digitizing (drawing) features on the map:

- Points
- Lines
- Polygons
- Rectangles
- Circles

These tools are implemented in `mapDraw.js` and integrated with the mapping widget. Users can create, edit, and delete geometric features.

Sources: `viewers/map/mapDraw.php` 92-115 `viewers/map/mapping.js` 80-92

Styling and Symbology

The system provides comprehensive styling options for map features:

- Marker symbols (circle, square, triangle, etc.)

- Line styles (color, width, dashing)
- Fill styles (color, pattern, opacity)
- Size and scale options
- Thematic styling based on attributes

Styles can be defined at the layer level or for individual features.

Sources: `viewers/map/mapping.js` 73-77 `viewers/map/mapLayer2.js` 378-397

Advanced Features

Timeline Integration

The Mapping System integrates with a timeline component, allowing users to visualize temporal data. Features on the map can be linked to time periods or specific dates, and users can filter the map by time.

Sources: `viewers/map/mapping.js` 60-61 `hclient/widgets/viewers/app_timemap.js` 22-235

Story Maps

The Story Map feature combines maps with narratives, allowing users to create interactive stories with geographical components. A story map consists of:

- A main map display
- A narrative with text, images, and other media
- A sequence of locations or features that the narrative refers to
- Controls to navigate through the story

Story maps are implemented in `app_storymap.js` and build on the core mapping components.

Sources: `hclient/widgets/viewers/app_storymap.js` 22-97

`hclient/widgets/viewers/app_storymap.js` 124-165

CMS Integration

The Mapping System integrates with the Heurist CMS system, allowing maps to be embedded in websites. The CMS widget configuration dialog (`editCMS_widgetCfg.js`) provides options for configuring maps within CMS pages.

Sources: `hclient/widgets/cms/editCMS_WidgetCfg.js` 214-345

`hclient/widgets/cms/editCMS_WidgetCfg.html` 394-714

Configuration Options

The Mapping System offers extensive configuration options, including:

Option	Description	Default
<code>map_rollover</code>	Show title of marker as tooltip	false
<code>map_popup_mode</code>	Show map info as control, popup, or none	'standard'
<code>notimeline</code>	Hide timeline component	false
<code>nocluster</code>	Disable marker clustering	false
<code>controls</code>	Comma-separated list of controls to show	'legend,bookmark,geocoder'
<code>basemap</code>	Initial base map to display	'OpenStreetMap'
<code>style</code>	Default style for features	(dynamic)
<code>zoomToPointInKM</code>	Zoom level for points (in km)	5

These options can be set when initializing the mapping widget or modified at runtime.

Sources: `viewers/map/mapping.js` 118-156

`hclient/widgets/cms/editCMS_WidgetCfg.html` 394-477

Map Document Structure

Map documents have a hierarchical structure:

- Map Document (contains multiple layers)
 - Layer 1 (has data source and style)
 - Data Source (query, file, tiles, etc.)
 - Style (color, size, icon, etc.)
 - Layer 2
 - Data Source
 - Style
 - Layer 3
 - Data Source

- Style

This structure is managed by the Map Document component and displayed in the Map Manager control panel.

Sources: `viewers/map/mapDocument.js` 160-200 `viewers/map/mapManager.js` 505-754

Integration with Search System

The Mapping System integrates with the Heurist search system, allowing users to visualize search results on maps. This integration works through:

1. A search widget that sends search queries
2. The search system that processes queries and returns results
3. The mapping system that displays the results on the map

Users can perform searches and see the results displayed geographically, and can also select features on the map to highlight the corresponding records in search results.

Sources: `viewers/map/mapping.js` 845-862 `hclient/widgets/search/searchInput.js` 230-273

Conclusion

The Heurist Mapping System provides a comprehensive solution for visualizing and interacting with geographical data. Its modular architecture, built on the Leaflet library, offers flexibility and extensibility while maintaining ease of use. The system's integration with other Heurist components like search, timeline, and CMS enhances its utility as a core feature of the platform.

Sources: `viewers/map/mapping.js` 1-112 `hclient/widgets/viewers/app_timemap.js` 1-21

`hclient/widgets/viewers/app_storymap.js` 1-21